

C++ FUNDAMENTALS

Dr. S. Vinila Jinny



Inline Functions

- improves the execution time and speed of the program.
- Normally, during function calls
 - its memory address is stored by the compiler
 - function arguments are copied on the stack
- Becomes an Overhead if function is very small, so we use inline concept here.
- An inline function is a function that is expanded in line when it is invoked.
- i.e., compiler replaces the function call with the corresponding function code.



Inline Functions

- Syntax:

```
inline function header
{
    function body
}
```

Example:

```
inline double cube(double a)
{
    return(a*a*a)
}
```

```
int main()
{
    cout<<cube(3);
    cout<<cube(2.5+1.5);
}
```



Default Arguments

- Call a function without specifying all its arguments
- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value.



Default Arguments

```
#include <iostream>
using namespace std;
int sum(int x, int y, int z = 0, int w = 0)
{
    return (x + y + z + w);
}
int main()
{
    cout << sum(10, 15) << endl;
    cout << sum(10, 15, 25) << endl;
    cout << sum(10, 15, 25, 30) << endl;
    return 0;
}
```



Class and objects

- Class binds together data and its associated functions together.
- Class Specification has two parts:
 1. Class declaration
 2. Class function definitions
- Class declaration describes the type and scope of its members.
- Class function definition describe how the class functions are implemented.



General form of Class

```
class class_name
{
    private:
        Variable declaration;
        Function declaration;
    public:
        Variable declaration;
        Function declaration;
};
```

- By default, the members of a class are private.



Example of a Class

```
class item
```

```
{
```

```
    int number;
```

```
    float cost;
```

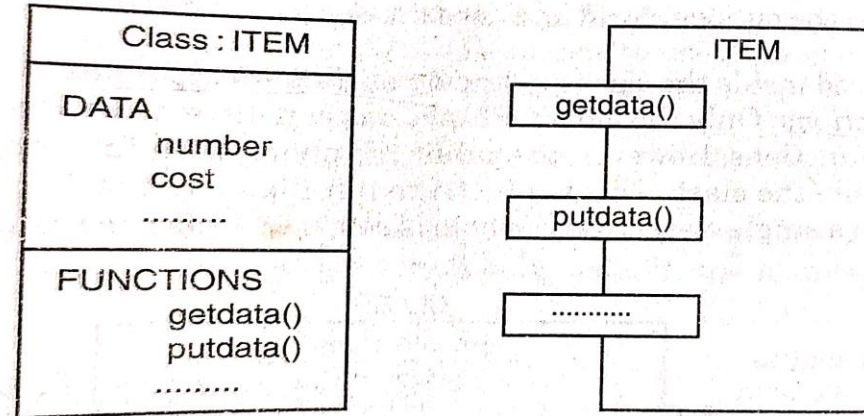
```
public:
```

```
    void getdata(int a, float b);
```

```
    void putdata();
```

```
};
```

```
item x; // creation of object
```



Accessing data members and member functions

- The data members and member functions of class can be accessed using the dot('.') operator with the object.

```
#include <iostream>
using namespace std;
class Student {
    public:
        int id;//data member (also instance variable)
        string name;//data member(also instance variable)
};
int main() {
    Student s1; //creating an object of Student
    s1.id = 201;
    s1.name = "Sonoo Jaiswal";
    cout<<s1.id<<endl;
    cout<<s1.name<<endl;
    return 0;
}
```



Member functions

- There are 2 ways to define a member function:
- Inside class definition
- Outside class definition
- To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.



Member function inside class

```
#include <iostream>
```

```
class Student {
```

```
public:
```

```
    int id;//data member (also instance variable)
```

```
    string name;//data member(also instance variable)
```

```
    void insert(int i, string n)
```

```
    {
```

```
        id = i;
```

```
        name = n;
```

```
    }
```

```
    void display()
```

```
    {
```

```
        cout<<id<<" "<<name<<endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Student s1; //creating an object of Student
```

```
    Student s2; //creating an object of Student
```

```
    s1.insert(201, "Sonoo");
```

```
    s2.insert(202, "Nakul");
```

```
    s1.display();
```

```
    s2.display();
```

```
    return 0;
```

```
}
```



Member function outside class

```
#include <iostream>
```

```
class Student {
```

```
    public:
```

```
        int id;//data member (also instance variable)
```

```
        string name;//data member(also instance variable)
```

```
        void insert(int i, string n);
```

```
void display()
```

```
{
```

```
    cout<<id<<" "<<name<<endl;
```

```
}
```

```
};
```

```
void Student :: insert(int i, string n)
```

```
{
```

```
    id = i;
```

```
    name = n;
```

```
}
```

```
int main() {
```

```
    Student s1; //creating an object of Student
```

```
    Student s2; //creating an object of Student
```

```
    s1.insert(201, "Sonoo");
```

```
    s2.insert(202, "Nakul");
```

```
    s1.display();
```

```
    s2.display();
```

```
    return 0;
```

```
}
```



Inline Concept Visited

- All the member functions defined inside the class definition are by default **inline**,
- But we can also make any non-class function inline by using keyword inline with them.
- Inline functions are actual functions, which are copied everywhere during compilation, like pre-processor macro, so the overhead of function calling is reduced.

